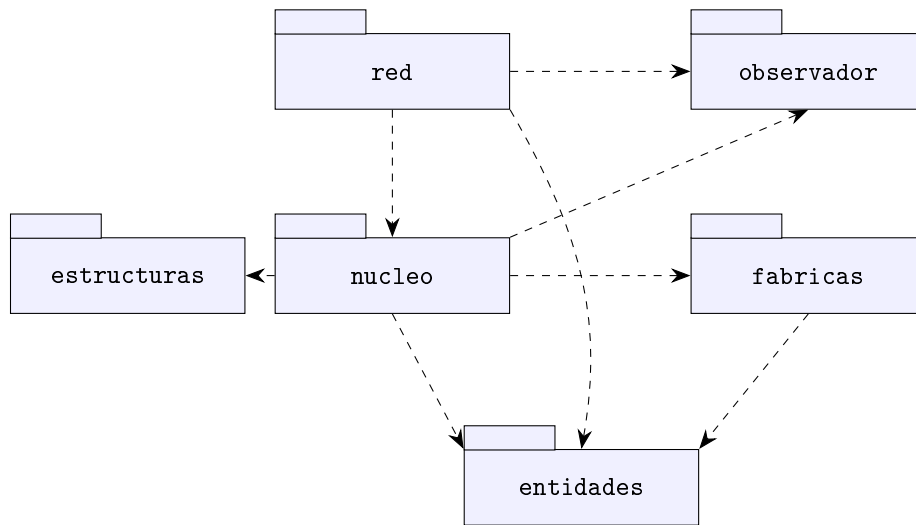


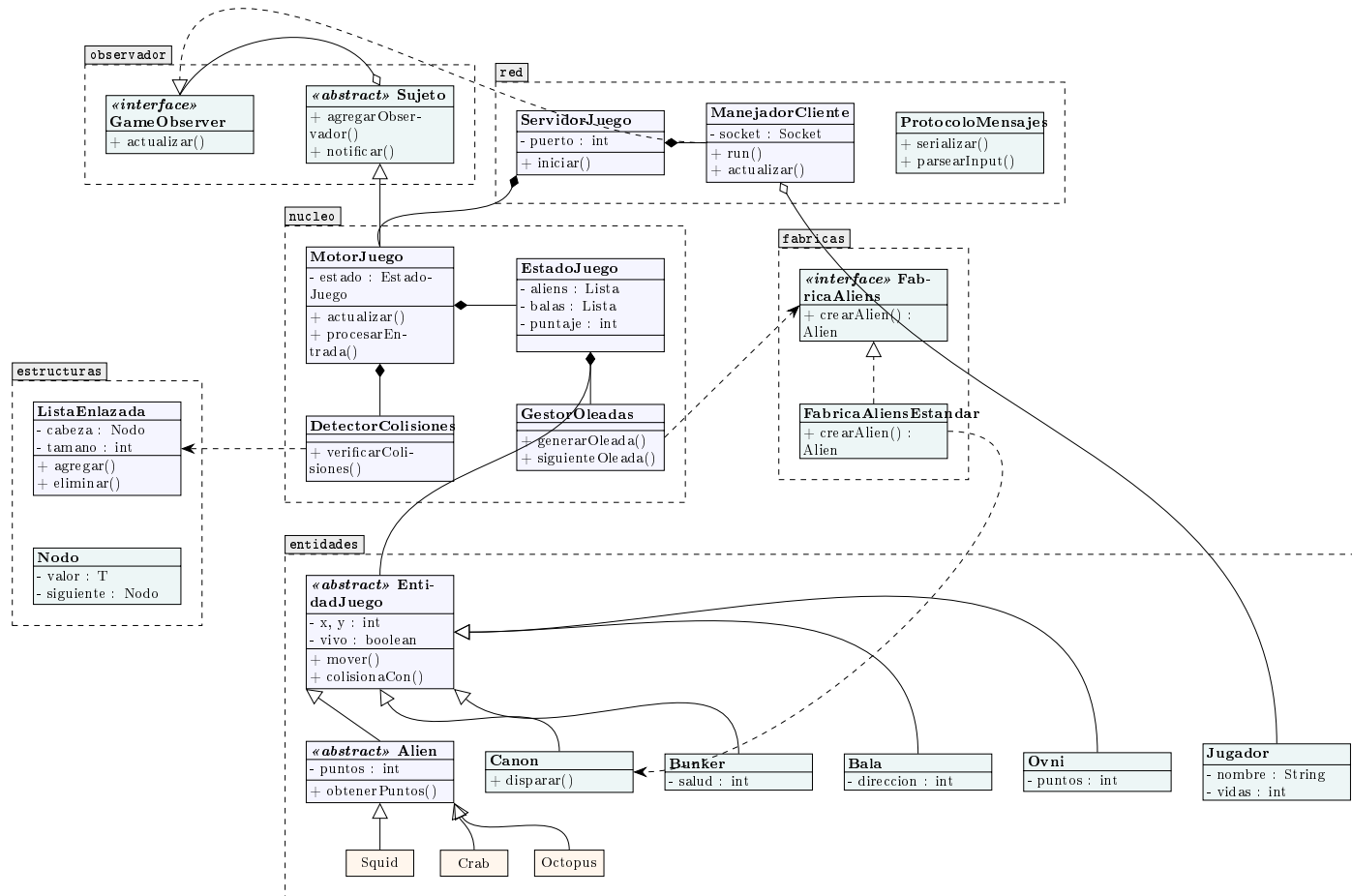
Diagramas del proyecto spaCEinvaders

1. Diagrama de paquetes



Lectura. La flecha discontinua indica dependencia: **red** depende de **nucleo**, **observador** y **entidades**; **nucleo** depende de **entidades**, **fabricas**, **observador** y **estructuras**; **fabricas** depende de **entidades**. Las dependencias apuntan hacia paquetes más estables (las entidades y las estructuras de datos no dependen de nadie).

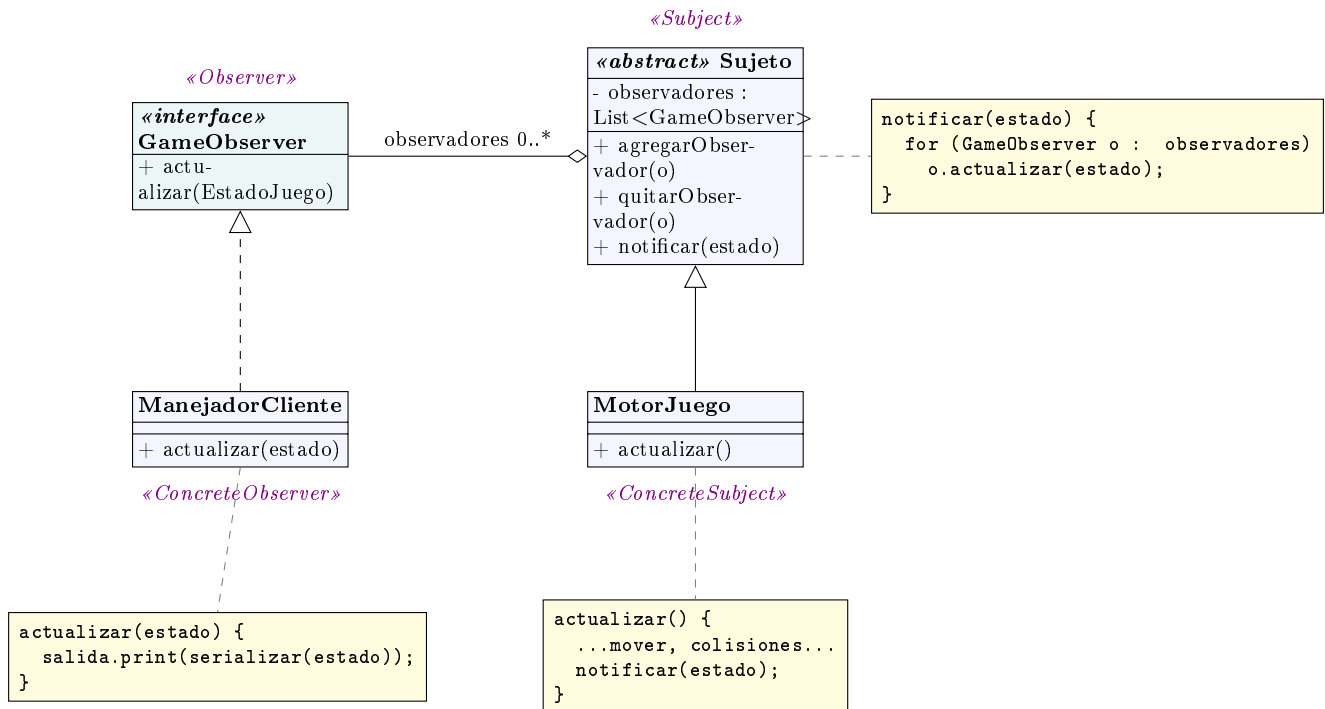
2. Diagrama de clases por paquete



Leyenda. Triángulo blanco continuo = herencia; triángulo blanco discontinuo = realización de interfaz; rombo negro = composición; rombo blanco = agregación; flecha discontinua = dependencia. El marco punteado con pestaña delimita cada **package** de Java.

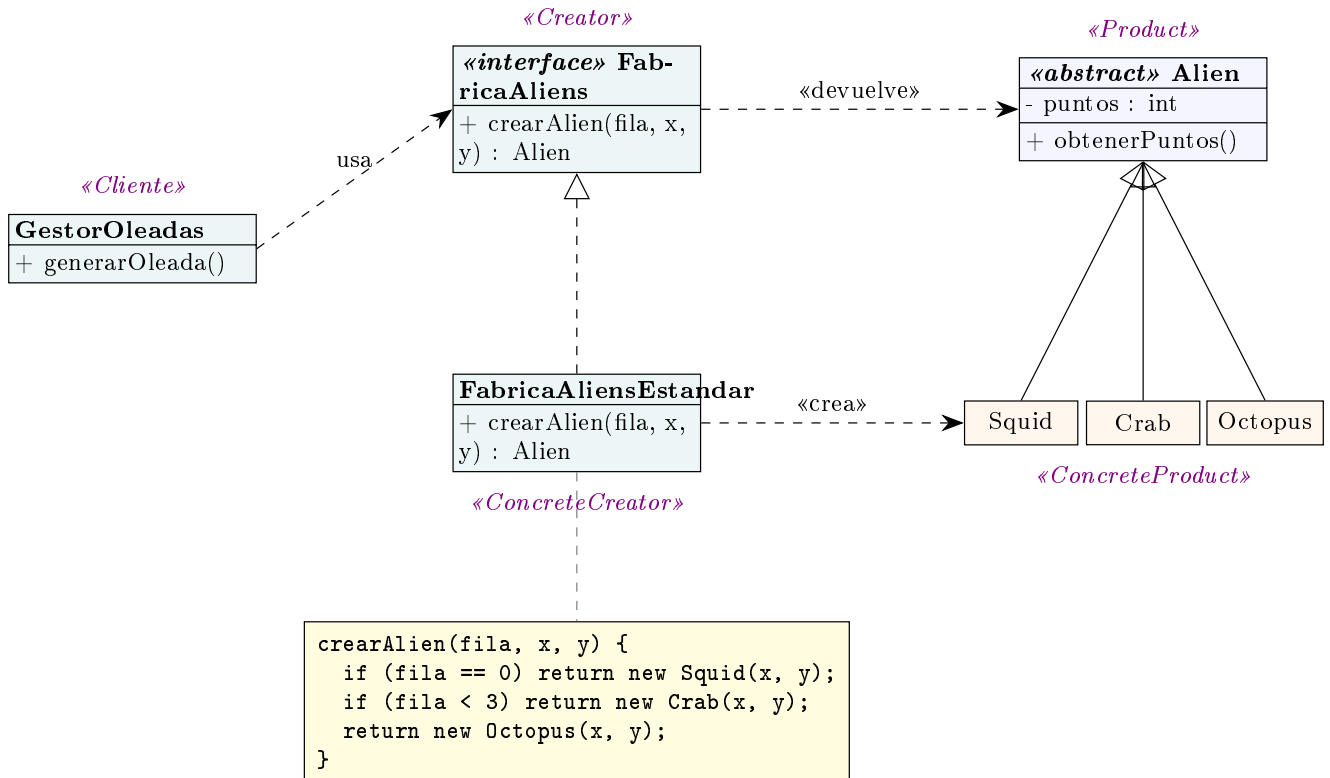
3. Implementación de los patrones de diseño

3.1 Patrón Observer (paquete observador)



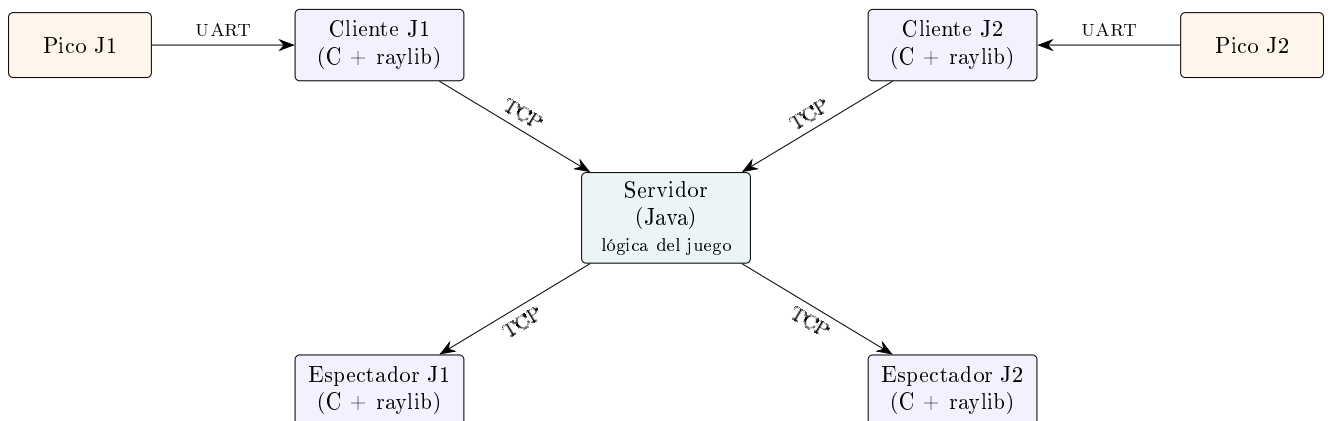
Cómo se implementa. **MotorJuego** (*ConcreteSubject*) hereda de **Sujeto**, que guarda la lista de observadores. En cada tick, `actualizar()` llama a `notificar(estado)`, que recorre la lista y llama `actualizar(estado)` en cada **ManejadorCliente** (*ConcreteObserver*); este serializa el estado y lo envía a su cliente. Conectar o desconectar clientes no toca la lógica del juego: solo se agregan o quitan observadores.

3.2 Patrón Factory Method (paquete fabricas)

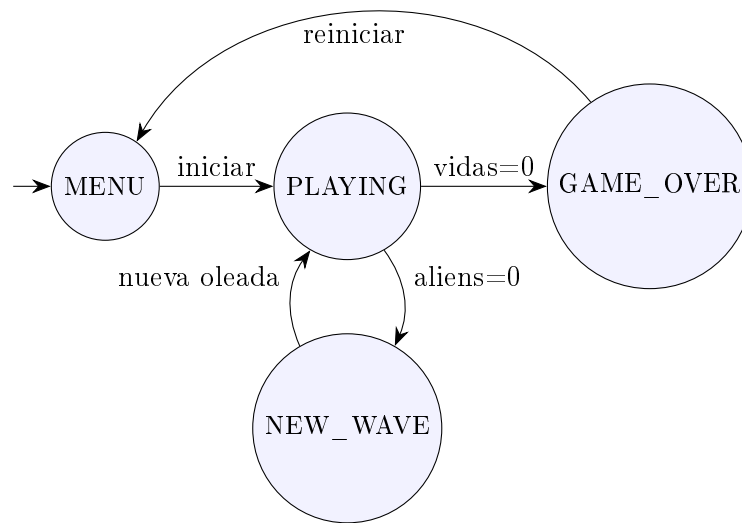


Cómo se implementa. `GestorOleadas` (*Cliente*) pide aliens por la interfaz `FabricaAliens` (*Creator*) sin conocer las clases concretas. `FabricaAliensEstandar` (*ConcreteCreator*) implementa `crearAlien()` y decide qué *ConcreteProduct* (`Squid`, `Crab` u `Octopus`) instanciar según la fila. Para agregar un tipo nuevo de alien solo se modifica la fábrica, no el resto del juego.

4. Diagrama de arquitectura



5. Diagrama de estados del juego



6. Diagrama de secuencia de mensajes

